

MeetUp!

Microservice Communication Styles

A guide for choosing how our services talk to each other

End-of-semester Spring Boot project — internal team reference

Why Communication Style Matters

In a microservices system, services need to talk to each other constantly — to fetch data, trigger actions, and stay in sync. The way they communicate has a huge impact on how complex the system is to build, debug, and maintain.

There are three main styles we need to understand: REST, gRPC, and event-driven messaging (Kafka / RabbitMQ). None of them is the best choice for every situation. This document explains what each one is, when to use it, and which design makes most sense for MeetUp! at our current stage.

Bottom line up front: REST alone is enough to build a working, impressive MeetUp! for an end-of-semester project. Kafka adds architectural realism. gRPC is a bonus. Read on to understand the tradeoffs so we can make an informed team decision.

Option 1 — REST over HTTP

What it is

REST (Representational State Transfer) uses standard HTTP verbs — GET, POST, PUT, PATCH, DELETE — with JSON payloads. When Service A needs something from Service B, it makes an HTTP request and waits for the response. This is the same pattern used to build regular web APIs, so every team member already knows it.

How it works in MeetUp!

Every service exposes a REST API. The API Gateway routes client requests to the right service. Services can also call each other directly — for example, the AI Service calls the Context Service to write a meeting summary after generating it.

Request	Caller	Callee	Why REST fits
Get pre-meeting briefing	Frontend client	Context Service	Client needs to wait for the data before rendering
Validate JWT token	API Gateway	Auth Service	Gateway must wait for yes/no before forwarding
Write AI summary	AI Service	Context Service	One-off write, needs confirmation it succeeded
Confirm draft task	Frontend client	Task Service	User action needs immediate acknowledgment
Get assignee suggestions	Frontend client	AI Service	Synchronous — user is waiting to see results

Pros and cons

Advantages	Disadvantages
Simple — every Spring Boot developer knows it	Tight coupling — if B is down, A's request fails
Easy to test with Postman or curl	One caller, one response — no fan-out to multiple services
Straightforward error handling (HTTP status codes)	Harder to scale under high load without load balancing
No extra infrastructure to set up	Polling required if you want near-real-time updates
Fast to build — annotations do most of the work	

Spring Boot implementation

- @RestController + @RequestMapping on each service
- RestTemplate or WebClient for service-to-service calls
- Spring Cloud OpenFeign for cleaner declarative HTTP clients between services
- Spring Cloud Gateway handles routing and JWT forwarding

Recommended approach for our MVP: build everything in REST first. It is the fastest path to a working system. We can introduce Kafka for specific high-value flows (like meeting.ended fan-out) once the core is working.

Option 2 — gRPC

What it is

gRPC is a high-performance remote procedure call framework developed by Google. Instead of JSON over HTTP/1.1, it uses Protocol Buffers (protobuf) — a compact binary format — over HTTP/2. You define your API in a .proto file, and gRPC generates both the client and server code automatically.

It is still request-response like REST, but faster and with strict type contracts enforced at compile time rather than at runtime.

How a .proto definition looks

```
// context.proto
service ContextService {
  rpc GetBriefing (BriefingRequest) returns (BriefingResponse);
}
message BriefingRequest { string meeting_id = 1; }
message BriefingResponse { string summary = 1; repeated string decisions = 2; }
```

From this file, Spring Boot generates both the server stub and the client stub. No manual JSON parsing, no mismatched field names.

Where gRPC fits in MeetUp!

Call	Benefit of gRPC over REST
Meeting Service → Tweening Service (get group suggestions)	Low latency, high frequency during live meetings
API Gateway → Auth Service (token validation)	Every request goes through this — speed matters
Task Service → Tweening Service (check group state)	Internal call with a strict contract — protobuf enforces it

Pros and cons

Advantages	Disadvantages
Faster than REST — binary protocol, HTTP/2 multiplexing	More setup — need protobuf compiler and generated code
Strongly typed contracts — breaking changes caught at compile time	Harder to test manually — no Postman equivalent (Kreya or grpcurl needed)
Auto-generated client code — no manual HTTP calls to write	Less familiar to most developers

Supports streaming (server-streaming, bidirectional)

Overkill for most calls in an end-of-semester project

Spring Boot implementation

- `grpc-spring-boot-starter` library (net.devh)
- Define `.proto` files in `src/main/proto/`
- Maven/Gradle plugin compiles `.proto` to Java classes automatically
- Annotate service with `@GrpcService`, inject with `@GrpcClient`

Our recommendation: skip gRPC for the MVP. It adds setup complexity without changing what the system can do. If we have time after the core is working, we could swap the API Gateway → Auth Service call to gRPC as a demonstrable optimisation.

Option 3 — Event-Driven Messaging (Kafka / RabbitMQ)

What it is

Instead of Service A calling Service B and waiting, A publishes an event to a message broker (Kafka or RabbitMQ). Any service that cares about that event subscribes and processes it independently — at their own pace, in parallel, even if A has already moved on.

This decouples services completely: A does not know or care who consumes its event. B can be down and catch up later. Five services can react to the same event simultaneously.

Kafka vs RabbitMQ — which one?

	Kafka	RabbitMQ
Model	Distributed log — events are retained	Message queue — consumed and deleted
Throughput	Very high (millions/sec)	High (tens of thousands/sec)
Setup complexity	Higher — needs Zookeeper / KRaft	Simpler — single broker
Message replay	Yes — consumers can re-read history	No — once consumed, gone
Best for	High-volume streams (transcripts, analytics)	Task queues, notifications, RPC-style events
Spring Boot support	@KafkaListener, KafkaTemplate	@RabbitListener, RabbitTemplate
For our project	Overkill but architecturally impressive	Simpler, faster to set up — recommended

Where event-driven fits in MeetUp!

Event-driven messaging is most valuable when one action needs to trigger multiple services simultaneously — a fan-out pattern that REST cannot do elegantly.

Event	Publisher	Consumers (all parallel)	Why not REST?
meeting.ended	Meeting Service	Context Service, AI Service, Tweening Service	REST would need 3 sequential calls — one failure breaks the chain
task.created	Task Service	Tweening Service, Context Service, AI Service	Same fan-out problem — 3 independent reactions
task.completed	Task Service	Tweening Service, Context Service, AI Service	Group dissolution + archival + weight update all independent

transcript.chunk	Meeting Service	Context Service, AI Service	High frequency during meeting — async is the right model
group.dissolved	Tweening Service	Context Service, Meeting Service, AI Service	Three services need to react independently

Pros and cons

Advantages	Disadvantages
Full decoupling — publisher does not know its consumers	Extra infrastructure — Kafka or RabbitMQ must run alongside services
Fan-out is trivial — one event, many consumers	Harder to debug — no single request trace across services
Resilience — consumer can be down and catch up	Eventual consistency — services may be briefly out of sync
Async processing — publisher is not blocked waiting	More complex error handling — dead letter queues, retries
Architecturally impressive for a project demo	

Spring Boot implementation

- RabbitMQ: spring-boot-starter-amqp dependency
 - `@RabbitListener(queues = "meeting.ended")` on consumer method
 - `rabbitTemplate.convertAndSend("meeting.ended", event)` to publish
 - Runs in Docker alongside services — one container
- Kafka: spring-kafka dependency
 - `@KafkaListener(topics = "meeting.ended")` on consumer method
 - `kafkaTemplate.send("meeting.ended", event)` to publish
 - Needs Kafka + Zookeeper (or KRaft) in Docker Compose

Our recommendation: use RabbitMQ for the fan-out events (meeting.ended, task.created, task.completed, group.dissolved). Use REST for everything else. This gives us the architectural story without Kafka's setup overhead.

Design Variations for MeetUp!

Below are three concrete designs your team can choose between — from simplest to most architecturally complete. All three produce a working, demonstrable system.

Variation A — REST Only (Simplest)

Every service exposes a REST API. Services call each other directly via HTTP. The API Gateway routes client requests. No message broker. No generated code.

Aspect	Detail
Complexity	Low — one pattern for everything
Setup time	Minimal — no extra infrastructure
Best for	Teams new to microservices, tight deadlines
Weak point	Fan-out calls (meeting.ended → 3 services) are sequential and brittle
Spring Boot stack	RestTemplate / OpenFeign, Spring Cloud Gateway, Spring Security (JWT)
Databases	PostgreSQL (Meeting, Task, User), MongoDB (Context), Neo4j (Tweening), Redis (cache)

How fan-out works in REST-only

When a meeting ends, the Meeting Service calls Context Service, then AI Service, then Tweening Service — one at a time. If any call fails, the rest do not happen. This is manageable at demo scale; handle it with retry logic and it is fine.

Variation B — REST + RabbitMQ (Recommended)

REST for synchronous queries and direct writes. RabbitMQ for events that trigger multiple consumers. This is the recommended design for MeetUp!— it cleanly separates the two communication needs and is straightforward to implement in Spring Boot.

Aspect	Detail
Complexity	Medium — two patterns, clear rules for which to use
Setup time	One extra Docker container (RabbitMQ)

Best for	Teams comfortable with Spring Boot, want architectural depth
Strength	Fan-out events are clean and resilient — services fully decoupled
Spring Boot stack	spring-boot-starter-amqp, OpenFeign, Spring Cloud Gateway
Databases	PostgreSQL, MongoDB, Neo4j, Redis

The rule for which to use

- Use REST when: one caller needs one answer, or a client is waiting for a response
- Use RabbitMQ when: one event should trigger multiple independent services

Interaction	Style	Reason
Client → any service	REST	Client needs a response
API Gateway → Auth (JWT check)	REST	Must wait for yes/no
AI Service → Context (write summary)	REST	Needs write confirmation
meeting.ended → Context + AI + Tweening	RabbitMQ	Fan-out to 3 consumers
task.created → Tweening + Context + AI	RabbitMQ	Fan-out to 3 consumers
task.completed → Tweening + Context + AI	RabbitMQ	Fan-out to 3 consumers
transcript.chunk → Context + AI	RabbitMQ	High-frequency async stream

Variation C — REST + RabbitMQ + gRPC (Full Design)

The full architecture. REST for client-facing APIs, RabbitMQ for event fan-out, gRPC for high-frequency internal service-to-service calls where performance and strict contracts matter.

Aspect	Detail
Complexity	High — three patterns, setup overhead for each
Setup time	Significant — .proto compilation, gRPC stubs, RabbitMQ
Best for	Teams who want to showcase distributed systems expertise

Strength	Optimal communication for each interaction type
Spring Boot stack	spring-kafka or spring-amqp, grpc-spring-boot-starter, OpenFeign
Databases	PostgreSQL, MongoDB, Neo4j, Redis

What moves to gRPC

- API Gateway → Auth Service: token validation on every request — speed matters
- Meeting Service → Tweening Service: group lookups during live meetings
- Task Service → Tweening Service: group state checks

Side-by-Side Summary

	Variation A REST only	Variation B REST + RabbitMQ	Variation C REST + RabbitMQ + gRPC
Setup complexity	Low	Medium	High
Extra infrastructure	None	RabbitMQ container	RabbitMQ + gRPC stubs
Fan-out events	Sequential REST calls	Clean, decoupled	Clean, decoupled
Internal call speed	Standard HTTP	Standard HTTP	Fast binary gRPC
Debugging ease	Easy	Medium	Harder
Architectural depth	Basic	Strong	Advanced
Risk for deadline	Very low	Low	Medium-high
Our recommendation	Fallback option	Go with this	If time allows

Team Decision

We recommend Variation B: REST for synchronous calls, RabbitMQ for fan-out events. This clearly separates the two communication concerns, keeps debugging manageable, and tells a strong architectural story in the demo — without the risk of gRPC setup eating into build time.

To implement this cleanly, the team should agree on one rule from day one:

Use REST when...

One service needs one answer, or a client is waiting for a response

Use RabbitMQ when...

One action should trigger multiple independent services simultaneously

If gRPC is something the team wants to explore, it can be introduced for a single internal call pair as a stretch goal after the core system is working — without affecting the overall architecture.